create overly granular roles or depend on exceptions—both of which undermine the clarity that RBAC is intended to provide. While roles can be managed in a declarative manner using infrastructure as code, RBAC remains fundamentally a static model. As systems become more complex, many organizations discover that they require more adaptive and fine-grained approaches.

5.2.4 Where RBAC still makes sense

Despite its limitations, RBAC remains a reliable choice in many settings. Its structure and predictability are especially beneficial when roles and responsibilities are stable and when access needs closely align with organizational boundaries.

RBAC works well in:

- Enterprise systems with clearly defined job roles—such as `finance-analyst`, `hr-admin`, or `sales-manager`, as long as fine-grained authorization isn't required.
- Internal tools used by employees whose access needs are linked to their job functions and remain consistent over time.
- Regulated environments, where auditability and separation of duty constraints are critical.
- Identity platforms and IAM tools that consider RBAC a foundational capability are often employed to manage access to SaaS applications.
- Infrastructure systems, where roles like `db-admin` or `cloud-ops` align clearly with operational responsibilities and can be managed as code.

RBAC serves as a practical foundation for organizations that are starting to formalize their approach to authorization. It offers structure without requiring a complete policy engine or a transition to attribute- or relationship-based models. For many teams, particularly those with limited authorization needs or a small number of applications, RBAC strikes the right balance of simplicity and control.